

COMP442 Assignment 3

Charles-Philippe La Richeliere

March 2024

Grammar

This is the attribute grammar, the semantic actions are included as symbols A,B,C ... Z, with the definitions of the semantic actions at the end of the grammar. We put curly braces around the symbol for the semantic action for clarity in the grammar below.

This is the attribute grammar,

Note that our `create_leaf()` function will make sure that we obtain the actual value of the token that it precedes, the lexeme of the token as it is seen in the source code that is being parsed, this is not done explicitly by the function because we do not pass a parameter, we use the value of the lookahead.

There are two main functions used for constructing the subtrees: `create_subtree_n_nodes(type, number of nodes)`, where `type` is the type of the node that is being created and `number of nodes` is the number of nodes to pop off the stack to create the subtree (eg. `number of children` for the node of type). The second is `create_subtree_pop_until(type,sentinel)` where `type` is the type of the parent node of the tree, and `sentinel` is some sentinel value node in the stack that we must pop until for the tree. The implementations of these are shown below in the design section.

Note that the semantic actions are just a form of pseudo-code the actual implementations follow this pseudo-code.

Attribute Grammar

START -> PROG eof {Y'} .

ADDOP -> plus {S'}
 | minus {T'}
 | or {U'} .

APARAMS -> EXPR REPTAPARAMS1 {V'}
 | .

APARAMSTAIL -> comma EXPR .

ARITHEXPR -> TERM RIGHTRECARITHEXPR .
 ARRAYSIZE -> lsqbr Flsqbr .
 Flsqbr -> intlit {B} rsqbr
 |rsqbr {H}.
 ASSIGNOP -> equal .

 EXPR -> ARITHEXPR OPTRELEXPR .
 OPTRELEXPR -> RELOP ARITHEXPR {U} | .

 FACTOR -> id {A} FACTOR2 REPTVARIABLEORFUNCTIONCALL
 |intlit {B}
 |floatlit {C}
 |lpar ARITHEXPR rpar
 |not FACTOR {S}
 |SIGN FACTOR {T}.

 FACTOR2 -> lpar {D} APARAMS {I} rpar {G} | {D} REPTIDNEST1 {E} {F} .

 FPARAMS -> {D} id {A} colon TYPE {D} REPTFPARAMS3 {P'} REPTFPARAMS4 {Q'}
 |.

FPARAMSTAIL -> comma id {A} colon TYPE {D} REPTFPARAMSTAIL4 {P'} .
 FUNCBODY -> lcurbr {D} REPTFUNCBODY1 rcurbr {L'}.
 FUNCDECL -> FUNCHEAD semi .
 FUNCDEF -> FUNCHEAD FUNCBODY .
 FUNCHEAD -> func id {A} lpar FPARAMS rpar arrow RETURNTYPE .
 REPTVARIABLEORFUNCTIONCALL -> IDNEST REPTVARIABLEORFUNCTIONCALL {X} | .
 IDNEST -> dot id {A} IDNEST2 .
 IDNEST2 -> lpar {B} APARAMS {I} rpar {G} | {D} REPTIDNEST1 {E} {F} .
 IMPLDEF -> {B} impl id {A} lcurbr REPTIMPLDEF3 rcurbr {O}.
 INDICE -> lsqbr ARITHEXPR rsqbr .

```

MEMBERDECL -> FUNCDECL {P}
            |VARDECL {Q} .

MULTOP -> mult {M'}
        |div {N'}
        |and {O'} .

OPTSTRUCTDECL2 -> inherits id {A} {B} REPTOPTSTRUCTDECL22 :
                |.

PROG -> REPTPROG0 .

RELEXPR -> ARITHEXPR RELOP ARITHEXPR {U} .

RELOP -> eq {E'}
        |neq {F'}
        |lt {G'}
        |gt {H'}
        |leq {I'}
        |geq . {J'}

REPTAPARAMS1 -> APARAMSTAIL REPTAPARAMS1
                |.

REPTFPARAMS3 -> ARRAYSIZE REPTFPARAMS3
                |.

REPTFPARAMS4 -> FPARAMSTAIL {R'} REPTFPARAMS4
                |.

REPTFPARAMSTAIL4 -> ARRAYSIZE REPTFPARAMSTAIL4
                |.

REPTFUNCBODY1 -> VARDECLORSTAT REPTFUNCBODY1
                |.

REPTIDNEST1 -> INDICE REPTIDNEST1
                |.

REPTIMPLDEF3 -> FUNCDEF REPTIMPLDEF3
                |.

REPTOPTSTRUCTDECL22 -> comma id {A} REPTOPTSTRUCTDECL22
                    |.

```

```

REPTPROG0 -> STRUCTORIMPLORFUNC REPTPROG0
    |.

REPTSTATBLOCK1 -> STATEMENT REPTSTATBLOCK1
    |.

REPTSTRUCTDECL4 -> VISIBILITY MEMBERDECL REPTSTRUCTDECL4
    |.

REPTVARDECL4 -> ARRAYSIZE REPTVARDECL4
    |.


RETURNRTYPE -> TYPE
    |void {X'} .

RIGHTRECARITHEXP ->
    |ADDOP {W'} TERM RIGHTRECARITHEXP .

RIGHTRECTERM ->
    | MULTOP {W'} FACTOR RIGHTRECTERM .

SIGN -> plus {A'}
    |minus {B'} .

STATBLOCK -> lcurbr REPTSTATBLOCK1 rcurbr
    |STATEMENT
    |.

STATEMENT -> id {A} STATEMENTBODY
    |if lpar RELEXPR rpar then STATBLOCK else STATBLOCK semi {Z'}
    |while lpar RELEXPR rpar STATBLOCK semi {A''}
    |read lpar VARIABLE rpar semi {B''}
    |write lpar EXPR rpar semi {C''}
    |return lpar EXPR rpar semi . {D''}

STATEMENTBODY -> {D} REPTIDNEST1 {E} {F} CHECKFORENDVAR
    | lpar {D} APARAMS {I} rpar {G} CHECKFORENDFUNC .

VARFUNCSTATENEST -> dot id {A} VARFUNCSTATENEST2 .

VARFUNCSTATENEST2 -> lpar {D} APARAMS {I} rpar {G} CHECKFORENDFUNC
    | {D} REPTIDNEST1 {E} {F} CHECKFORENDVAR .

```

```

CHECKFORENDFUNC -> {Y} semi
                  | {Z} VARFUNCSTATENEST .

CHECKFORENDVAR -> {Y} ASSIGNOP EXPR semi {E''}
                  | {Z} VARFUNCSTATENEST .

STRUCTDECL -> {B} struct id {A} OPTSTRUCTDECL2 lcurbr REPTSTRUCTDECL4 rcurbr semi {R} .

STRUCTORIMPLORFUNC -> STRUCTDECL
                    | IMPLDEF
                    | FUNCDEF .

TERM -> FACTOR RIGHTRECTERM .

TYPE -> integer {J}
      | float {K}
      | id . {L}

VARDECL -> let id {A} colon TYPE {B} REPTVARDECL4 {V} semi .

VARDECLORSTAT -> VARDECL {W}
                | STATEMENT .

VARIABLE -> id {A} VARIABLE2 .

VARIABLE2 -> {D} REPTIDNEST1 {E} {F} REPTVARIABLE .
VARIABLE2 -> lpar {D} APARAMS {I} rpar {G} dot id {A} VARIABLE2 {X} .

REPTVARIABLE -> dot id {A} VARIABLE2 {X}| .

VISIBILITY -> public {C'}
             | private {D'} .

```

Semantic Actions

And the corresponding semantic actions in pseudo-code,

```

//This creates a leaf node of type id and value of id

A{
    push_node(create_leaf_node(id));

```

```

}

//This creates a leaf token for an intlrit

B{
    push_node(create_leaf_node(intlrit),stack);
}

//This creates a leaf token a floatlit

C{
    push_node(create_lead_node(floatlit),stack);
}

//This creates a sentinel token for us to later call create_subtree_pop_until(), provides /
D{
    push_node(create_leaf_node($));
}

//This creates a dimlist, this is the actual values that will be passed different from an ar

E{
    push_node(create_subtree_pop_until(dimlist,$));
}

//This creates a var node, var node will have
//two children an id node with the name and an dimlist node.

F{
    push_node(create_subtree_n_nodes(var, 2));
}

//This creates a funccall node, will have two
//children an id node with the name of the function and an aparams list
//a list of arguments passed to the function.

G{
    push_node(create_subtree_n_nodes(funccall, 2));
}

//This is a leaf node for an emptydim

H{
    push_node(create_leaf_node(emptydim));
}

```

```

//This creates the aparams list node that will be a child of a funccall node.

I{
  push_node(create_subtree_pop_until(aparams list, $));
}

//This creates a node of type integer, note that this is different from intlit.

J{
  push_node(create_leaf_node(integer));
}

//This creates a node of type float
K{
  push_node(create_leaf_node(float));
}

//Note that this creates a different kind of
leaf token that has a type of type and the correct id for the lexeme.
//This is a node that signifies a type, not a variable or function name.

L{
  push_node(create_leaf_node(id,type));
}

//This creates an inherits node which will be a child of a struct node.

N{
  push_node(create_node_pop_nuntil(inherits,$));
}

//This will create the impl node, we have any number of
//funcdef nodes that can be a child of impl
//nodes so we use pop_until version of create_subtree.

O{
  push_node(create_subtree_pop_until(impl,$));
}

//This creates a funcdecl node, a funcdecl must have 4 children.

P{
  push_node(create_subtree_n_nodes(funcdecl, 4));
}

```

```

//This creates a vardecl node with 4 children, note that we give this
//4 children because we have to include the visibility where this
//semantic action gets called as well.

Q{
push_node(create_subtree_n_nodes(vardecl,4));
}

//This creates a struct node, a struct can have any number of functions declarations and va
//so we use the pop_until version.

R{
push_node(create_subtree_pop_until(struct,$));
}
//We create a not node, not will have a single child a factor.

S{
push_node(create_subtree(not,1));
}
//We create a sign node , note that we have two children,
//the sign either + or - and the expression to apply it to.

T{
push_node(create_subtree(sign,2));
}

//This creates the relop node with three children an
//arithmetic expression a relative operation token
//and an arithmetic expression token, thus we have 3 children.

U{
push_node(create_subtree_n_nodes(relop, 3));
}

//This creates the node for the array dimensions
//aggregates the numbers and makes them child nodes,
//can have any number of arraydimensions.

V{
push_node(create_subtree_pop_until(arraydims, $));
}

//This creates a vardecl node, we have 3 nodes
//here because a vardecl in a function definition

```

```

//will only have 3 children as we don't need the visibility.

W{
  push_node(create_subtree_n_nodes(vardecl,3));
}

//This creates a node that represents the dot operator.

X{
  push_node(create_subtree_n_nodes(dot,2));
}

//This semantic action is more complex, the way that assignment
//statement work is whenever we pass be a dot we create a var/funccall
//node and make that node a child of a dot and push the dot onto the stack.
//So when we arrive at the assignment operator we have a stack that has
//many dots on it but we need a single expression, so we go back up the
//stack and make the current dot/var a child of the dot that is the top
//of the stack, we set the current to be that dot and push it onto the
//stack, we do this until we have reached the sentinel token.

Y{
  top = pop_node();
  while( peek() == dot):
    next_dot = pop_node();
    next_dot.add_child(top);
    top=next_dot();
  endwhile.

  if(peek() == $)
    pop();

  push_node(top);
}

//This will create a dot for the previously mentioned algorithm,
//used whenever we are creating assignment statement with a
//nested class objects.

Z{
  push_node(create_subtree_n_nodes(dot,1));
}

//Creates a leaf node for plus operator

A'{

```

```

    push_node(create_leaf_node(plus));
}

//Creates a leaf node minus operator

B'{
push_node(create_leaf_node(minus));
}

//creates a leaf node for public visibility

C'{
push_node(create_leaf_node(public));
}

//creates a leaf node for private visibility

D'{
push_node(create_leaf_node(private));
}

//creates a leaf node for equality operator

E'{
push_node(create_leaf_node(eq));
}

//Create a leaf node for the not equal operator

F'{
push_node(create_leaf_node(neq));
}

//create a leaf node for the less than or equal operator.

G'{
push_node(create_leaf_node(leq));
}

H'{
push_node(create_leaf_node(geq));
}

I'{
push_node(create_leaf_node(lt));
}

```

```

}

J'{
    push_node(create_leaf_node(gt));
}

//create a funcdef node, will always have 4 children, the name,
//the fparams, the return type and a funcbody(same as statblock)

K'{
    push_node(create_subtree_n_nodes(funcdef,4));
}

//creates a funcbody node which is the same as a statblock node.

L'{
    push_node(create_subtree_n_nodes(funcbody,$));
}

M'{
    push_node(create_leaf_node(mult));
}

N'{
    push_node(create_leaf_node(div));
}

O'{
    push_node(create_leaf_node(and));
}

//This creates an array dims node, this is meant to be
//used for variable declarations and function declarations

P'{
    push_node(create_subtree_pop_until(arraydims,$));
}

//This creates a fparams list node which will have as children all
//fparams of the current function, can have any number of children.

Q'{
    push_node(create_subtree_pop_until(fparams list,$));
}

```

```

//An fparam node will always have 3 children the name the type and arraydimensions

R'{
push_node(create_subtree_n_nodes(fparam, 3));
}

//Note that the type for these nodes will be the same as
//the operator name, this will differ from those that we
//get using sign which will have a type of sign instead of lets plus.

S'{
push_node(create_leaf_node(plus));
}

T'{
push_node(create_leaf_node(or));
}

U'{
push_node(create_leaf_node(minus));
}

V'{
push_node(create_node_pop_until(aparams list, $));
}

//This creates an operator node(*,+, -, /), we do this by
//popping the 3 top nodes off the stack, the middle one will be
//the operator the other two the factors. We then place these back
//in the same order and we create an operator subtree of the type
//of the operator that we popped off the stack.

W'{
factor1 = pop_node();
operator = pop_node();
factor2=pop_node();

//we get the type of the operation this way.
type=operator->type;

//Push the factors back on the stack.
push_node(factor2);
push_node(factor1);

//create and push the subtree

```

```

push_node(create_subtree_n_nodes(type,2));

}

//This creates a void type node, note that we
//can only use this as the return type of a function
X'{
    push_node(create_leaf_node(void));
}

//This creates a prog node, at the end of parsing we should only have
//nodes of type funcdef, impldef and structdecl, we organize these into
//lists and make a node that has as children these lists this is what
//this function is doing.

Y'{
    funcdefstack = init_stack();
    structdeclstack=init_stack();
    impldefstack = init_stack();

    while(stack is not empty):
        temp = pop_node();
        if temp.type == structdecl then:
            push_node_on(structdeclstack, temp);
        elif temp.type == funcdef then:
            push_node_on(funcdefstack);
        elif(temp.type == impldef) then:
            push_node_on(impldefstack);
        else
            problem should not be here....

    structdecllist = create_subtree_pop_until_empty(structdeclstack);
    impldeflist = create_subtree_pop_until_epmty(impldefstack);
    funcdeflist = create_subtree_pop_until_empty(funcdefstack);

    push_node(structdecllist);
    push_node(impldeflist);
    push_node(funcdeflist);

    //We create the final prog node, which will have three children, a
    //node for the fundef nodes, a node for the impldef nodes and
    //a node for the structdecl nodes.
    node *ast_root = create_subtree_n_nodes(prog, 3));

}

```

```
//Creates an ifnode, an ifnode will
//always have 3 children a relexpr a then statblock and
//an else statblock.
```

```
Z'{
push_node(create_subtree_n_nodes(ifnode,3));
}
```

This creates whilenode, while will have 2 nodes
as children a relexpr node and a statblock

```
A'{
push_node(create_subtree_n_nodes(whilenode,2));
}
```

The last 3 semantic actions create a read,write
and return node all of these will have 1 node
as a child an EXPR or VARIABLE

```
B'{
push_node(create_subtree_n_nodes(read, 1));
}
```

```
C'{
push_node(create_subtree_n_nodes(write,1));
}
```

```
D'{
push_node(create_subtree_n_nodes(return,1));
}
```

This creates the assignop node, the assignop has two children the assignment
value an expr node and the assignee, when we reach the end of
CHECKFORENDVAR the top of the stack is guaranteed to
have these two nodes.

```
E'{
push_node(create_subtree_n_nodes(assignop, 2));
}
```

Design

The original parser was built using the Top Down Recursive Descent method. To augment the parser to build the AST we use an explicit stack that we push nodes of the tree onto. The node data structure is as follows,

```
typedef struct node {
    struct node **children;
    int numchildren;
    struct node *right_sibling;
    struct node *left_sibling;
    struct node *parent;
    const char *type;
    const char *value;
    int node_number;
} node;
```

Each node has a link to its left sibling and right sibling, if applicable, if not these links are NULL. Then each node has a link to its parent node, if applicable to the node if not it is set to NULL. If the node is not a leaf node it will have an array of pointers to child nodes, if the node has no children this node will be set to NULL.

Each node has two values that are associated with it, the value of the node which is the actual lexeme of the token in the source code and the type which represents the syntactic category and semantic meaning of the node. Leaf nodes will have both a value and a type whereas other nodes will typically only have a type and no value unless we have an operator (*,/,+,-,!) etc.

We have several functions that we use to create the subtrees that will make up the AST, we have `create_subtree_pop_until()`. This function creates a subtree by popping nodes until we reach some sentinel node in the stack.

We also have the `create_subtree_n_nodes`, this function will take the type of a node, the number of nodes to pop off of the stack and create a subtree where the type of the root is the type passed and the children are the nodes that get popped off of the stack.

These are the implementations of these functions, they can be found in the `AST_gen.c` file and the function headers can be found in the header file `AST_gen.h`.

This function creates a family of nodes where the root of the tree is the node that is passed (parent) and the type is passed as a string.

```

int create_family(node *parent, node **children, int numchildren,
                  const char *type) {
    if (children == NULL) {
        parent->numchildren = 0;
        parent->children = NULL;
        parent->node_number = x++;
    }
    parent->type = type;
    parent->children = children;
    parent->numchildren = numchildren;
    parent->node_number = x++;
    for (int i = 0; i < numchildren; i++) {
        children[i]->parent = parent;
        if (i == 0) {
            children[i]->left_sibling = children[numchildren - 1];
            children[i]->right_sibling = children[i + 1];

        } else if (i == (numchildren - 1)) {
            children[i]->left_sibling = children[i - 1];
            children[i]->right_sibling = children[0];
        } else {
            children[i]->left_sibling = children[i - 1];
            children[i]->right_sibling = children[i + 1];
        }
    }
    return 1;
}

```

This function pops n nodes off of the stack and returns the nodes as an array of pointers to the nodes.

```

node **pop_n_nodes(int len, semantic_stack *stack) {
    if (!stack) {
        error("ERROR - pop_n_nodes(): Stack has not been allocated any memory "
              "exiting");
        return NULL;
    }
    if (len <= 0) {
        error("ERROR - pop_n_nodes(): Cannt pop zero or less nodes from the stack "
              "exiting");
        return NULL;
    }
    if (len > stack->size) {
        error("ERROR - pop_n_nodes(): Cannot pop more nodes than there are nodes "

```

```

        "on the stack terminating");
    return NULL;
}

node **nodes = malloc(sizeof(node *) * len);

for (int i = 0; i < len; i++) {
    nodes[i] = pop_node(stack);
}

return nodes;
}

```

This is the implementation for the `create_subtree_n_nodes()`.

```

node *create_subtree_n_nodes(const char *type, int len, semantic_stack *stack) {

    node *parent = init_node(type);
    node **nodes = pop_n_nodes(len, stack);

    if (create_family(parent, nodes, len, type)) {
        return parent;
    }
    return NULL;
}

node *create_subtree_pop_until(const char *type, const char *sentinel,
                               semantic_stack *stack) {

    if (!stack) {
        error(
            "ERROR - create_subtree_pop_until() : Stack has not been allocated any "
            "memory exiting.");
        return NULL;
    }

    // This function should work correctly.
    int len = 0;
    node **nodes = pop_until(sentinel, stack, &len);

    node *parent = malloc(sizeof(node));
    if (create_family(parent, nodes, len, type)) {
        return parent;
    }
}

```

```

    }

    return NULL;
}

```

The testing for the parser is done using the files in the tests/parser/. We have two main files that we have provided additionally for testing, the positivetesting.src and negativetesting.src files. the positivetesting.src file contains only syntactically valid structures and the negative testing file contains only structures that should not be in the syntactical specification of the language.

This function takes a parent node and a child node, it makes the child node a child of the parent node, makes it the leftmost child of the node.

```

void add_child(node *parent, node *child) {
    if (!parent || !child) {
        error("add_child_as_first: One of the nodes is NULL");
        return;
    }

    node **new_children =
        realloc(parent->children, sizeof(node *) * (parent->numchildren + 1));
    if (!new_children) {
        error("add_child_as_first: Memory allocation failed");
        return;
    }
    parent->children = new_children;

    for (int i = parent->numchildren; i > 0; i--) {
        parent->children[i] = parent->children[i - 1];
    }

    parent->children[0] = child;
    child->parent = parent;

    if (parent->numchildren > 0) {
        child->right_sibling =
            parent->children[1]; // The former first child is now the second
        parent->children[1]->left_sibling = child;
    }
    child->left_sibling =
        parent->children[parent->numchildren]; // Assuming circular sibling links;
}

```

```

// adjust as needed

if (parent->numchildren == 0) {
    child->left_sibling = child;
    child->right_sibling = child;
}

parent->numchildren += 1;
}

```

We use these functions to augment the parser and insert the semantic actions into the RHS of the relevant productions, to see the actual semantic actions view the subsection semantic actions.

The parser will output the AST in form a DOT script that can be used in a viewer to get the graphical representation of the AST for the source code that has just been parsed.

Tools

The main tool that was used for this section of the project is the graphviz online DOT viewer. This was used to look at the trees produced from the parser and verify their correctness. Minor modifications were also made to the grammar to make it more straightforward to insert semantic actions into the RHS of productions, to do this the ucalgary tool was used again the same from assignment 2.